



UNIVERSIDADE KIMPA VITA

INSTITUTO POLITÉCNICO

CURSO DE LICENCIATURA EM ENGENHARIA INFORMÁTICA

TRABALHO DE LÓGICA DE PROGRAMAÇÃO II

TEMA: SISTEMA DE GESTÃO DE DADOS DE ESTUDANTE

POR:

Afonso Garcia José

António Manuel João

Ndombaxi Ranel

Vicente Capitão

Maria Estevão Francisco

Docente: Moyo

UÍGE, 2025/2026

Indice

1. Introducao
 2. Objetivos de Aprendizagem
 3. Analise de Requisitos
 4. Algoritmos Implementados
 - 4.1 Algoritmos de Ordenacao
 - 4.2 Algoritmos de Pesquisa
 5. Estrutura do Programa
 6. Manual de Utilizacao
 - 6.1 Compilacao no Dev-C++
 - 6.2 Execucao e Interacao
 7. Ficheiros do Projeto
 8. Resultados e Testes
 9. Conclusao
-

1. Introducao

O presente relatorio descreve o desenvolvimento de um Sistema de Gestao de Dados de Estudantes em linguagem C, elaborado no ambito da disciplina de Logica de Programacao II. O trabalho tem como finalidade consolidar competencias fundamentais em programacao estruturada, com enfase na implementacao de algoritmos classicos de ordenacao e pesquisa, bem como na manipulacao segura de ficheiros.

A gestao de dados de estudantes constitui uma necessidade recorrente no ambiente universitario. Este projeto simula um sistema simples e funcional que permite ler, organizar, pesquisar e persistir informacoes relativas a estudantes, servindo como base pratica para a compreensao de conceitos essenciais que todo engenheiro informatico deve dominar.

2. Objetivos de Aprendizagem

O objetivo geral deste trabalho e capacitar o estudante a desenvolver um programa em C que integre algoritmos de ordenacao, pesquisa e operacoes sobre ficheiros. Os objetivos especificos sao:

- **Implementar algoritmos de ordenacao:** Bubble Sort, Selection Sort e Insertion Sort, aplicados a diferentes criterios (matricula, nome e nota).
- **Implementar algoritmos de pesquisa:** Pesquisa Linear e Pesquisa Binaria, demonstrando a diferenca de eficiencia e os requisitos de cada metodo.
- **Manipular ficheiros em C:** Leitura e escrita em ficheiros de texto (.txt) e binarios (.bin), com tratamento de erros de I/O.
- **Desenvolver funcoes modulares:** Estruturar o codigo em funcoes coesas e reutilizaveis, promovendo boa manutenibilidade.
- **Aplicar estruturas de dados basicas:** Uso de arrays e structs para representar e organizar os dados dos estudantes.

3. Analise de Requisitos

3.1 Requisitos Funcionais

Funcionalidade	Descricao
Leitura de dados	Ler estudantes de estudantes.txt (matricula, nome, nota)
Ordenacao	Ordenar por matricula (crescente), nome (alfabetico) ou nota (decrescente)
Pesquisa	Pesquisar por matricula (linear/binaria) ou por nome (linear)
Escrita em ficheiro	Gravar a lista em saida.txt (texto) ou estudantes.bin (binario)
Menu interativo	Interface de linha de comando clara com todas as opcoes acessiveis

3.2 Requisitos Nao Funcionais

- **Usabilidade:** Menu intuitivo com mensagens informativas ao utilizador.
- **Confiabilidade:** Tratamento de erros de I/O (ficheiro inexistente, erro de escrita).
- **Manutenibilidade:** Codigo modular, comentado, com nomes de funcoes e variaveis significativos.
- **Desempenho:** Algoritmos correctamente implementados com logica adequada para o contexto.

➤ Algoritmos Implementados

3.3 Algoritmos de Ordenacao

Foram implementados tres algoritmos classicos de ordenacao. Cada um e aplicado a um criterio diferente, permitindo ao utilizador comparar o seu comportamento.

Algoritmo	Criterio	Ordem	Complexidade
Bubble Sort	Numero de Matricula	Crescente	$O(n^2)$
Selection Sort	Nome do Estudante	Alfabetica	$O(n^2)$

Insertion Sort	Nota Final	Decrescente	$O(n^2)$
----------------	------------	-------------	----------

Exemplo — Bubble Sort (por Matricula):

```
for (i = 0; i < total - 1; i++) { for (j = 0; j < total - 1 - i; j++) { if
(lista[j].matricula > lista[j+1].matricula) { temp = lista[j]; lista[j]
= lista[j+1]; lista[j+1] = temp; }
}
}
```

➤ Algoritmos de Pesquisa

Foram implementados dois algoritmos de pesquisa distintos, com diferentes requisitos e eficiencias:

Algoritmo	Criterio	Requisito	Complexidade
Pesquisa Linear	Matricula ou Nome	Nenhum	$O(n)$
Pesquisa Binaria	Matricula	Lista ordenada por matricula	$O(\log n)$

Exemplo — Pesquisa Binaria:

```
int inicio = 0, fim = total - 1, meio; while (inicio <= fim) { meio =
(inicio + fim) / 2; if (lista[meio].matricula == matricula) return
meio; else if (lista[meio].matricula < matricula) inicio = meio +
1; else fim = meio - 1; } return -1; /* nao encontrado */
```

➤ Estrutura do Programa

O programa e organizado num unico ficheiro **main.c**, estruturado em funcoes modulares. A estrutura de dados central e a **struct Estudante**:

```
typedef struct
{ int
matricula;
char
nome[100];
```

```
float nota; }
Estudante;

Estudante lista[100]; /* array global de estudantes */int totalEstudantes = 0;
```

Funcao	Descricao
lerFicheiroTexto()	Le estudantes.txt para o array lista[]
lerFicheiroBinario()	Le dados de estudantes.bin (formato binario)
gravarFicheiroTexto()	Grava a lista em saida.txt
gravarFicheiroBinario()	Grava a lista em estudantes.bin
bubbleSortMatricula()	Ordena por matricula — Bubble Sort
selectionSortNome()	Ordena por nome — Selection Sort
insertionSortNota()	Ordena por nota — Insertion Sort
pesquisaLinearMatricula()	Pesquisa linear por numero de matricula
pesquisaBinariaMatricula()	Pesquisa binaria por matricula (requer ordenacao)
pesquisaLinearNome()	Pesquisa linear por nome do estudante
exibirLista()	Imprime todos os estudantes na consola
menuOrdenacao()	Sub-menu de escolha do algoritmo de ordenacao
menuPesquisa()	Sub-menu de escolha do metodo de pesquisa

➤ Manual de Utilizacao

3.4 Compilacao no Dev-C++

1. Abrir o Dev-C++ e seleccionar **File** → **New** → **Source File**.
2. Copiar o conteudo do ficheiro **main.c** para o editor.
3. Guardar o ficheiro como **main.c** numa pasta dedicada ao projecto.
4. Copiar o ficheiro **estudantes.txt** para a **mesma pasta** onde esta o main.c.
5. Ir a **Tools** → **Compiler Options** e adicionar **-std=c99** nas flags adicionais.
6. Premir **F9** (ou **Execute** → **Compile & Run**) para compilar e executar.

➤ Execucao e Interacao

Apos a execucao, e apresentado o menu principal. Recomenda-se a seguinte sequencia de utilizacao para um teste completo:

Passo	Opcao no Menu	Accao
1	Opcao 1	Ler dados do ficheiro estudantes.txt
2	Opcao 3	Exibir a lista original
3	Opcao 4 → 1	Ordenar por Matricula (Bubble Sort)
4	Opcao 5 → 2	Pesquisa Binaria por matricula
5	Opcao 4 → 2	Ordenar por Nome (Selection Sort)
6	Opcao 4 → 3	Ordenar por Nota (Insertion Sort)
7	Opcao 6 → 1	Gravar a lista ordenada em saida.txt
8	Opcao 6 → 2	Gravar em formato binario (estudantes.bin)
9	Opcao 0	Sair do programa

4. Ficheiros do Projeto

Ficheiro	Tipo	Descricao
main.c	Codigo-fonte C	Programa completo com todos os algoritmos e funcoes
estudantes.txt	Texto (entrada)	10 estudantes de exemplo para teste do programa
saida.txt	Texto (saida)	Lista gerada pelo programa apos ordenacao/alteracao
estudantes.bin	Binario (saida)	Dados gravados em formato binario pelo programa
relatorio.pdf	Documento PDF	Este relatorio tecnico do projecto

Conteudo de estudantes.txt (dados de entrada):

```

1001 Joao_Silva 15.50
1002 Maria_Santos 18.00
1003 Pedro_Costa 12.30
1004 Ana_Ferreira 19.50
1005 Carlos_Gomes 10.00
1006 Fatima_Lopes 16.70
1007 Bruno_Mendes 14.20
1008 Sofia_Nunes 17.80
1009 Tiago_Ribeiro 11.50
1010 Beatriz_Alves 20.00

```

Conteudo de saida.txt (ordenado por Nota — Insertion Sort):

```

1010 Beatriz_Alves 20.00
1004 Ana_Ferreira 19.50
1002 Maria_Santos 18.00
1008 Sofia_Nunes 17.80
1006 Fatima_Lopes 16.70
1001 Joao_Silva 15.50
1007 Bruno_Mendes 14.20
1003 Pedro_Costa 12.30
1009 Tiago_Ribeiro 11.50

```


1005 Carlos_Gomes 10.00

5. Resultados e Testes

Foram realizados testes para validar cada funcionalidade do sistema. A tabela abaixo resume os resultados obtidos:

Teste	Resultado	Estado
Leitura de estudantes.txt (10 registos)	10 estudantes carregados correctamente	OK
Bubble Sort por Matricula	Lista ordenada de 1001 a 1010	OK
Selection Sort por Nome	Lista em ordem alfabetica (Ana → Tiago)	OK
Insertion Sort por Nota	Lista decrescente (20.0 → 10.0)	OK
Pesquisa Linear por Matricula 1006	Fatima_Lopes, Nota: 16.70 encontrada	OK
Pesquisa Binaria por Matricula 1004	Ana_Ferreira, Nota: 19.50 encontrada	OK
Pesquisa Linear por Nome Sofia_Nunes	Matricula: 1008, Nota: 17.80 encontrada	OK
Gravar saida.txt	Ficheiro gerado com 10 linhas correctas	OK
Gravar estudantes.bin	Ficheiro binario gerado com sucesso	OK
Abrir ficheiro inexistente	Mensagem de erro exibida, programa continua	OK

Conclusao

O desenvolvimento deste sistema permitiu consolidar e aplicar de forma pratica os conceitos fundamentais de Logica de Programacao II. A implementacao dos tres algoritmos de ordenacao — Bubble Sort, Selection Sort e Insertion Sort — demonstrou claramente as suas logicas internas e confirmou a sua equivalencia em termos de complexidade temporal $O(n^2)$ para o caso medio.

A comparacao entre a Pesquisa Linear e a Pesquisa Binaria revelou de forma pratica a importancia da ordenacao previa: enquanto a pesquisa linear percorre todos os elementos, a pesquisa binaria reduz o espaco de busca a metade a cada iteracao, sendo significativamente mais eficiente para listas de grande dimensao.

A manipulacao de ficheiros em formato texto e binario permitiu compreender as diferencas entre os dois formatos: os ficheiros de texto sao legiveis por humanos e facilmente editaveis, enquanto os binarios oferecem maior eficiencia de leitura/escrita e menor espaco de armazenamento. O tratamento de erros de I/O tornou o programa mais robusto e confiavel.

Este trabalho constituiu uma experiencia de aprendizagem enriquecedora, reforçando a importancia da programacao modular, da documentacao do codigo e da realizacao de testes sistematicos para garantir a qualidade do software.